

Vol 0: Kernel Foundations LENGUAJES Y RUNTIMES Sintaxis y PEPs Inclusión del módulo estándar ensurepip con la llegada de Python 3.4, formalizada bajo el PEP 453 para arrancar pip de forma nativa. 0x00	Vol 0: Kernel Foundations SEGURIDAD Y EXPLOITS Malware y Exploits Identificación del ataque a la cadena de suministro de SolarWinds, comprometiendo redes gubernamentales mediante actualizaciones firmadas. 0x01	Vol 0: Kernel Foundations SEGURIDAD Y EXPLOITS Malware y Exploits Divulgación de la vulnerabilidad crítica Shellshock en GNU Bash, presente en el analizador del intérprete durante un cuarto de siglo. 0x02
Vol 0: Kernel Foundations LENGUAJES Y RUNTIMES Frameworks Web Nacimiento de FastAPI, integrando Starlette, Pydantic y tipado estático con generación automática de esquemas OpenAPI. 0x03	Vol 0: Kernel Foundations SISTEMAS Y REDES Sistemas UNIX / Linux Debut del intérprete de comandos GNU Bash (Bourne-Again SHell), diseñado como el reemplazo libre del clásico shell de Stephen Bourne. 0x04	Vol 0: Kernel Foundations LENGUAJES Y RUNTIMES Sintaxis y PEPs Oficialización del módulo enum en la biblioteca estándar para enumeraciones robustas bajo el PEP 435. 0x05
Vol 0: Kernel Foundations SISTEMAS Y REDES Sistemas UNIX / Linux Incorporación de las tuberías (pipes) en Unix, permitiendo encadenar flujos de salida y entrada estándar entre programas. 0x06	Vol 0: Kernel Foundations SEGURIDAD Y EXPLOITS Desastres de Software Inmovilización de la flota Boeing 737 MAX tras dos siniestros aéreos causados por fallos en el software de control de vuelo. 0x07	Vol 0: Kernel Foundations CLOUD E INFRAESTRUCTURA Automatización IaC Publicación del paper sobre Chubby, el servicio de bloqueo distribuido con Paxos que sostiene los clústeres de Google. 0x08
Vol 0: Kernel Foundations SISTEMAS Y REDES Sistemas UNIX / Linux Anuncio fundacional del Proyecto GNU en Usenet para crear un sistema operativo completo y totalmente libre compatible con Unix. 0x09	Vol 0: Kernel Foundations ALMACENAMIENTO Y DATOS Motores de Bases de Datos Publicación del paper de Dynamo por Amazon, pionero en alta disponibilidad con consistencia eventual y particionamiento hash. 0x0A	Vol 0: Kernel Foundations HARDWARE Retrochips y GPUs Presentación del procesador Apple M1, marcando la transición de Mac hacia arquitectura ARM con alta eficiencia y memoria unificada. 0x0B
Vol 0: Kernel Foundations LENGUAJES Y RUNTIMES Sintaxis y PEPs Debut del emparejamiento de patrones estructurales (pattern matching) con sintaxis match/case vía PEP 634 en Python 3.10. 0x0C	Vol 0: Kernel Foundations ALMACENAMIENTO Y DATOS Motores de Bases de Datos Publicación del modelo relacional de datos por Edgar F. Codd, fundamentando las bases matemáticas de las bases de datos modernas. 0x0D	Vol 0: Kernel Foundations SEGURIDAD Y EXPLOITS Malware y Exploits Detección de una sofisticada puerta trasera (backdoor) infiltrada en el código fuente de la utilidad de compresión XZ Utils. 0x0E

Stéphane Chazelas

2014

Permitía ejecutar código arbitrario al procesar definiciones de funciones exportadas dentro de variables de entorno en servidores CGI.

0x02

FireEye / Atribuido a APT29 (SVR)

2020

Los atacantes insertaron la puerta trasera Sunburst directamente en el proceso de compilación de Orion, pasando inadvertidos durante meses.

0x01

Donald Stufft y Nick Coghlan

2014

Permitió inicializar pip sin conexión ni requerir descargas externas como get-pip.py, unificando la distribución de paquetes en Python.

0x00

Barry Warsaw y Eli Bendersky

2013

Sustituyó patrones informales basados en clases de constantes enteras o diccionarios, dotando a Python de tipos enumerados con identidad única.

0x05

Brian Fox (Free Software Foundation)

1989

El nombre es un juego de palabras con 'born-again' y Steve Bourne; con el tiempo se consolidó como el shell predeterminado en entornos Linux.

0x04

Sebastián Ramírez (tiangolo)

2018

Su interfaz de documentación interactiva basada en Swagger UI y OpenAPI aceleró significativamente el desarrollo y testeo de APIs.

0x03

Mike Burrows (Google)

2006

Burrows priorizó una interfaz simple de archivos y bloques gruesos (coarse-grained) para ocultar la complejidad matemática de Paxos.

0x08

Boeing

2019

El sistema MCAS se activó repetidamente al confiar en lecturas erróneas de un único sensor de ángulo de ataque, anulando la intervención manual.

0x07

D. McIlroy y K. Thompson (Bell Labs)

1973

McIlroy propuso el concepto y Thompson implementó la llamada de sistema en pocas horas, cimentando la modularidad de Unix.

0x06

Apple (Johnny Srouji)

2020

Integró CPU, GPU y memoria en un único empaquetado (SoC), logrando un rendimiento por vatio que superó a procesadores de consumo previos.

0x0B

G. DeCandia, W. Vogels et al. (Amazon)

2007

Diseñado para evitar carritos de compra caídos, priorizó la disponibilidad sobre la consistencia estricta, inspirando a Cassandra y Riak.

0x0A

Richard Stallman (MIT AI Lab)

1983

Su nombre es un acrónimo recursivo para 'GNU's Not Unix'; precedió a la creación de la FSF y concibió herramientas libres esenciales como GCC.

0x09

Andrés Freund (CVE-2024-3094)

2024

Freund descubrió la intrusión al investigar anomalías de consumo de CPU y latencia en sshd provocadas por libzma infectada.

0x0E

Edgar F. Codd (IBM Research)

1970

Codd propuso independizar la estructura lógica de los datos de su soporte físico, enfrentando reticencias en IBM por proteger su producto IMS.

0x0D

Brandt Bucher y Guido van Rossum

2021

Inspirado en Scala y Haskell, permite descomponer y validar estructuras de secuencias y objetos complejos más allá de un simple switch.

0x0C